

Different calendar } → Different
Different format } countries

Mutator & Accessor methods

Local Date now and date etc
↓
= new Date (get Date (now));

↓
It is not changed
instead it returns object

original
remain unchanged → Immutable

String s = "hello";
↓
processed to uppercase()
new String **HELLO**

HELLO

Method does not mutate
original

Georgian Calendar s = new Georgian
Calendar
[(999, 1112)]

s.plusDays(20)
↓
mutate

Methods which change state
of objects → mutator methods

Methods which get state
→ Accessor methods



car
type position → set

car() → car
position mutate → set
↓
mutator methods

Brand
getSomeBrand()

↓
we get some brand
name

But it does not mutate
(Accessors)

Custom class

class

{

}

→ You are now going
to create for your own
purpose

Instance fields → state of object

Constructors → to invoke object creation

method → behaviour of object



```
class Car {  
    private String brand;  
    private double price;  
    public void run() {  
        }  
}
```

```

class Car
{
    private String brand;
    private String color;
    private Tyre tyre;

    void run()
    {
        id.position = equals("green");
        Tyre.setposition("run");
    }
}

```

Instance →

method
change the
state of object

Workhouse classes

Simplicity

```

Employee
{
    id -> int (private)
    salary -> double (private)
}

```

```

class Tyre
{
    private int position;
    public void setposition(int pos)
    {
        position = pos;
    }
    public String getposition()
    {
        return position;
    }
}

```


class Employee {

private int id;
private String name;
private double salary;

} State

public int getId()
{
return id;
}

public String getName()
{
return name;
}

public Double getSalary()
{
return salary;
}

→ Behaviours

public void setId(int i)
{
id = i;
}

public void setName(String n)
{
name = n;
}

public void setSalary(double d)
{
salary = d;
}

public Employee (int i, String name,
double s)

{
id = i;
name = name;
salary = s;
}

}

multiple source files

multiple source files

you use multiple so you
can identify easily

Compile only necessary ones

{ private } no outside method
can read this

public → Everyone can see it

En Strongly recomm to privatization

Be aware!



If there is a chance
someone may or some
code may access

Constructors

Same name as class name

new Class A ();

Constructor always with new keyword

A(); → compile time error

Shadow error in Variables

Constructors or methods
local variables & instance ones have
same name local variable
over shadows it

Java 10

Var → local variable infer type

You must infer any
initial data type

Null reference

Null $\rightarrow$ refers it does not
associate with
any object yet

obj null
 $\downarrow$
does not contain object

If you try to access any Rs
from null $\rightarrow$ NullPointerException

Permissive
 $\downarrow$
give default
value

rough-love
 $\downarrow$
objects requires
non null str
()

$\downarrow$
Throw exception
even if passing
value is null

Implicit & Explicit

Methods

↓
↳ Instance fields

Implicit parameter

↳ this keyword

```
public Employee (String name, double salary)
```

```
{  
    this.name = name;  
    this.salary = salary;  
}
```

→ distinguish from local variable

this → Implicit parameter (Current object)

Explicit parameter → parameters
Explicitly present on method